

面向 LLM / Agent Systems 科研的深度学习零基础课

第 06 讲

实验设计：Batch、三维 Tensor 与共享 Linear

实验目标：用可复现实验回答一个核心问题——“对 (B, T, D) 直接做 Linear，是否等价于对每个 Token 单独使用同一组参数？”

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1 实验背景

本讲已经从理论上得到：

$$X : (B, T, D), \quad W : (D, O), \quad b : (O,)$$

可以直接计算：

$$Y = XW + b : (B, T, O)$$

但零基础学习不能只停留在“相信公式”。本实验要把这个结论拆成多个可验证的观察。

2 研究问题

1. 二维 Batch 一次计算，与逐样本循环是否数值完全一致？
2. 三维 (B, T, D) Tensor 直接 Linear 后，Shape 是否变成 (B, T, O) ？
3. 直接三维 Linear，是否等价于先 reshape 成 $(B \times T, D)$ 再恢复？
4. 是否等价于逐 Batch、逐 Token 循环？
5. Bias 是否被广播到每一个 Token 的输出？
6. Batch Size 和 Sequence Length 是否会改变 Linear 的参数数量？
7. PyTorch `nn.Linear` 是否与 NumPy 的结论一致？

3 实验假设

- H1: Batch 向量化结果与逐样本循环一致；
- H2: $(2, 3, 4) @ (4, 2) \rightarrow (2, 3, 2)$ ；
- H3: 3D 直接运算与 flatten $(6, 4)$ 再恢复完全一致；
- H4: 每个 Token 都共享同一组 W, b ；
- H5: Bias $(2,)$ 会加到每一个 (b, t) 的 2 维输出上；
- H6: 改变 B, T 不改变 $D \rightarrow O$ Linear 参数数量；
- H7: PyTorch 输出与 NumPy 数值一致。

4 变量设计

类别	设计
固定参数	$W : (4, 2)$ 、 $b : (2,)$
输入变量	二维 $X : (3, 4)$ ；三维 $X_3 : (2, 3, 4)$
对照方法	向量化矩阵运算 vs Python 循环 vs flatten/reshape vs PyTorch
主要观测	输出 Shape、数值是否 allclose、参数数量
通过标准	对照方法结果一致；Shape 与理论预测一致

5 实验环境

最低环境：

- Python 3.x；
- NumPy；
- PyTorch 可选，但建议安装后运行最后一个对照实验。

运行：

```
python lesson06_batch_tensor_experiment.py
```

6 实验 1：二维 Batch vs 逐样本循环

6.1 目的

验证 Batch 只是把多个相同形式的单样本运算向量化，而不是改变计算规则。

6.2 操作

计算：

$$Y_{batch} = XW + b$$

再逐样本：

$$y_i = x_iW + b$$

最后堆叠成 Y_{loop} 。

6.3 预期

$$Y_{batch} = Y_{loop}$$

程序中用 `np.allclose` 判断。

7 实验 2：构造微型 LLM Tensor

设置：

$$B = 2, \quad T = 3, \quad D = 4, \quad O = 2$$

因此：

$$X_3 : (2, 3, 4), \quad W : (4, 2)$$

预期：

$$Y_3 : (2, 3, 2)$$

运行前先预测

在看程序输出前，先写下：

1. $X_3[0, 0]$ 的 Shape；
2. $X_3[0, 0]@W + b$ 的 Shape；
3. Y_3 的 Shape。

答案应该分别是 (4,)、(2,)、(2, 3, 2)。

8 实验 3：Direct 3D vs Flatten

将：

$$(2, 3, 4)$$

reshape 为：

$$(6, 4)$$

然后做相同 Linear：

$$(6, 4)@ (4, 2) \rightarrow (6, 2)$$

再恢复：

$$(2, 3, 2)$$

若与直接 3D 结果一致，说明三维 Linear 可以理解为“6 个 Token 向量共享同一映射”。

9 实验 4：逐 Token 双层循环

使用：

```
for batch_index in range(B):  
    for token_index in range(T):
```

```
Y[batch_index, token_index] = \
    X[batch_index, token_index] @ W + b
```

与直接：

```
Y = X @ W + b
```

对照。

科研视角

这里训练的是科研中的“实现等价性验证”。当你不确定一个高层 API 究竟做什么时，构造一个极小输入，用更笨但透明的实现作为 reference，是非常实用的调试方法。

10 实验 5：Bias Broadcasting

先计算：

$$Z = XW$$

再：

$$Y = Z + b$$

观察：

$$Y - Z$$

是否在每个 Token 上都等于同一个 b 。

11 实验 6：参数量不依赖 B, T

对于：

$$D = 4, O = 2$$

参数量：

$$4 \times 2 + 2 = 10$$

无论 $B = 2, T = 3$ ，还是 $B = 128, T = 4096$ ，参数量都保持 10。

需要区分：

- 参数量：模型有多少可学习数字；
- 激活/计算量：一次输入有多少数据需要处理。

12 实验 7：PyTorch 对照

创建：

```
layer = nn.Linear(4, 2)
```

将参数手动设置为与 NumPy 相同，再输入：

$$X : (2, 3, 4)$$

预期输出：

$$(2, 3, 2)$$

且数值与 NumPy 一致。

13 实际结果摘要

本材料包已实际运行脚本，结果：

- 二维 Batch 与逐样本循环： **True**；
- 三维输入 (2, 3, 4) 输出： (2, 3, 2)；
- Direct 3D 与 flatten/restore： **True**；
- 逐 Token 双循环与直接运算： **True**；
- Bias 在所有 Token 上重复加同一向量；
- $4 \rightarrow 2$ 参数量恒为 10；
- PyTorch 与 NumPy： **True**。

14 如何记录实验结果

建议在学习笔记中建立如下表格：

实验	运行前预测	实际结果	是否理解原因
二维 Batch			
3D Linear			
Flatten 等价性			
Token 循环			
Bias Broadcasting			
PyTorch 对照			

15 实验结论

必须能自己复述

对 (B, T, D) 进行 $D \rightarrow O$ Linear，本质上就是对 $B \times T$ 个 D 维 Token 向量使用完全相同的 W, b 。框架把这些重复计算组织成高效的张量运算，因此输出为 (B, T, O) 。

16 科研方法迁移

本实验真正训练的不只是 Batch，而是一个可复用的科研/调试模式：

1. 先写出理论预测；
2. 构造极小、可手算的数据；
3. 写一个透明的 reference implementation；
4. 再与向量化/框架 API 对照；
5. 使用 `allclose` 等量化判断，而不是“看着差不多”；
6. 解释不一致来自 Shape、轴语义、Broadcasting 还是数值误差。